Guía de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Temporizador de hardware y generación de señal con ESP32
Nombre del Docente	Luis Darío Sinche C.
Fecha	30 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 3): 2 h presenciales y 1 h para terminar el informe
Modalidad de trabajo	Se trabaja y se entrega en parejas, con el acompañamiento del técnico docente; todos simulan en Wokwi y la prueba física se hace con la placa del grupo, si la tiene
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica:

Objetivo. Configurar un temporizador de hardware del ESP32 con la API del core 3.x, usar su interrupción para hacer parpadear un LED y generar una onda cuadrada de frecuencia calculada, y medir esa señal con un analizador lógico.

Al terminar, la pareja podrá:

- Calcular el valor de alarma para un tiempo dado.
- Explicar el recorrido completo: el contador llega a la alarma, se produce la interrupción, se ejecuta la ISR y `loop()` responde.
- Medir la frecuencia de una señal digital y compararla con la calculada.

3. Fundamento teórico

Temporizador de hardware. Un temporizador (timer) es un circuito del microcontrolador que cuenta pulsos de duración conocida, por lo que sirve para medir tiempo. Cada vez que el contador suma 1 se produce un tick; con 1 000 000 ticks por segundo, cada tick dura 1 µs. La alarma es el número de ticks al que debe llegar el contador para generar un aviso y, con autorrecarga, el contador vuelve a 0 y el aviso se repite. Fórmulas: resolución = 1 / f; tiempo = alarma × resolución; alarma = tiempo × f.

Interrupciones e ISR. Una interrupción es un aviso del hardware que hace que la CPU pause lo que hace, ejecute una función corta y vuelva al mismo punto. Esa función es la ISR (rutina de servicio de interrupción); en esta práctica se llama `alSonarAlarma` y es un callback: no se llama desde el programa, sino que se entrega al temporizador para que él la llame. `IRAM_ATTR` guarda la ISR en la RAM interna rápida, y `volatile` indica al compilador que la variable puede cambiar desde la ISR y que debe leerla siempre de la memoria. Con el patrón bandera, la ISR solo pone una variable en `true`; `loop()` la revisa, la baja y hace el trabajo lento.

API del temporizador en el core 3.x. Se usan tres funciones, siempre en este orden dentro de `setup()`: `timerBegin(1000000)` reserva un temporizador libre y fija 1 000 000 ticks por segundo (1 tick = 1 µs; el core calcula el prescaler por sí solo y el temporizador empieza a contar de inmediato); `timerAttachInterrupt(timer, alSonarAlarma)` entrega el callback; y `timerAlarm(timer, ALARMA_US, true, 0)` fija la alarma en ticks, con autorrecarga (`true`) y sin límite de repeticiones (`0`). La variable `hw_timer_t *timer` guarda la «manija» del temporizador reservado; si `timerBegin()` devuelve `NULL`, no se pudo reservar ninguno. La mayoría de tutoriales de internet usan la versión 2 del core (`timerBegin(0, 80, true)`, `timerAlarmWrite`, `timerAlarmEnable`), que no compila en la versión 3: usen solo los códigos de esta guía.

Onda cuadrada y el factor 2. Una onda cuadrada es una señal digital que cambia entre bajo (0 V) y alto (3,3 V en el ESP32) a ritmo constante, pasando el mismo tiempo en cada nivel. El periodo (T) es la duración de un ciclo completo y la frecuencia es f = 1 / T, en hercios (Hz); por ejemplo, T = 1 ms equivale a 1 kHz. En la Parte B la ISR invierte el pin en cada alarma y un ciclo necesita dos cambios: cada alarma produce medio periodo, el periodo de la señal es el doble del tiempo de alarma y su frecuencia es la mitad de la frecuencia de las alarmas. Con ticks de 1 µs: `ALARMA_US = 1 000 000 / (2 × f_señal)`; por ejemplo, 1 kHz → 500, 5 kHz → 100 y 100 Hz → 5000.

Analizador lógico de Wokwi. Registra solo niveles digitales (0 o 1) a lo largo del tiempo, como un osciloscopio que solo ve unos y ceros; tiene 8 canales (D0 a D7) y un pin GND que se conecta a tierra. Al detener la simulación, Wokwi descarga el archivo `wokwi-logic.vcd` (Value Change Dump: formato estándar que guarda en qué instante cambió cada señal), que se abre con PulseView o GTKWave. En PulseView: Open → flecha a la derecha → Import Value Change Dump data; las señales aparecen como logic.D0, logic.D1, etc.

4. Materiales y reactivos:

Hardware (por grupo, solo para la Parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard
1	LED rojo de 5 mm
1	Resistencia de 330 Ω
Varios	Cables jumper macho-macho

En Wokwi (todos) se usan el ESP32 DevKit V1, 1 LED rojo, 1 resistencia de 330 Ω y 1 analizador lógico (Logic Analyzer (8 channels)); el circuito completo ya viene en el diagram.json del Anexo A.

5. Equipos y herramientas

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE con el paquete esp32 by Espressif Systems versión 3.x, sin librerías externas, para la Parte C.
- PulseView (gratuito, sigrok.org) o GTKWave (gratuito), para abrir el archivo .vcd. Instálenlo antes de la práctica.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

Equipos (grupos con placa, Parte C):

- Osciloscopio del laboratorio o multímetro con modo de frecuencia (Hz), si está disponible.

6. Procedimiento / Metodología

Práctica de 2 h presenciales en parejas, con el acompañamiento del técnico docente: un temporizador de hardware del ESP32 hace parpadear un LED mediante su interrupción y luego genera una onda cuadrada que se mide con el analizador lógico de Wokwi. Todos trabajan en Wokwi; los grupos con placa hacen también la prueba en el ESP32 real. **Antes de la clase:** lean el Fundamento teórico e instalen PulseView.

Tabla de conexiones

Todos los pines elegidos son seguros: no son de arranque o strapping (0, 2, 5, 12, 15) ni solo de entrada (34–39).

Elemento	Pin del ESP32	Rótulo en la placa	Conexión
LED	GPIO19	D19	GPIO19 → resistencia 330 Ω → ánodo del LED (pata larga); cátodo (pata corta) → GND
Señal cuadrada	GPIO18	D18	Wokwi: canal D0 del analizador. Placa: punta del osciloscopio o multímetro
LED en el analizador	GPIO19	D19	Wokwi: canal D1 del analizador (permite ver también el LED)
Tierra	GND	GND	GND del analizador, cátodo del LED y referencia del instrumento

Cuidados: los pines del ESP32 trabajan a 3,3 V; no conecten 5 V a ningún GPIO. Con 330 Ω el LED rojo consume unos 4 mA, por debajo del límite recomendado de 10 mA por pin.

Parte A. LED con temporizador y patrón bandera (Wokwi, 30 min)

En esta parte el temporizador genera una alarma cada `ALARMA_US` microsegundos. La ISR solo levanta una bandera; `loop()` la ve, la baja, cambia el LED e imprime en el Monitor Serie el instante del cambio, medido con `millis()`.

A1. Entren a wokwi.com, creen un proyecto nuevo de **ESP32** y abran la pestaña diagram.json.

A2. Borren todo su contenido y peguen el Anexo A. Deben aparecer el ESP32, el LED, la resistencia y el analizador lógico ya cableados.

A3. Abran la pestaña sketch.ino, borren su contenido y peguen el Código 1.

A4. Antes de simular, predigan: con `ALARMA_US = 1000000`, ¿cada cuántos milisegundos cambiará el LED? Anoten la predicción en la Tabla A.

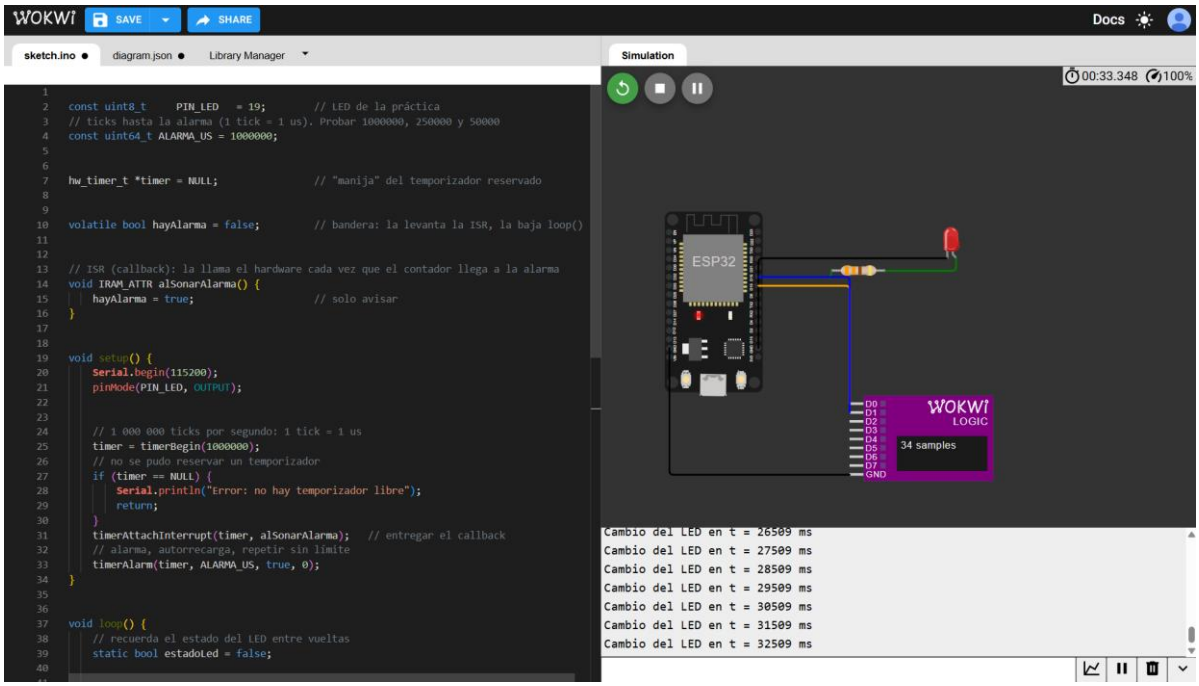
A5. Presionen ▶ (Start). Observen el LED y el Monitor Serie durante unos 10 segundos.

A6. Calculen la diferencia entre dos instantes consecutivos que imprime el Monitor Serie y anótenla en la Tabla A.

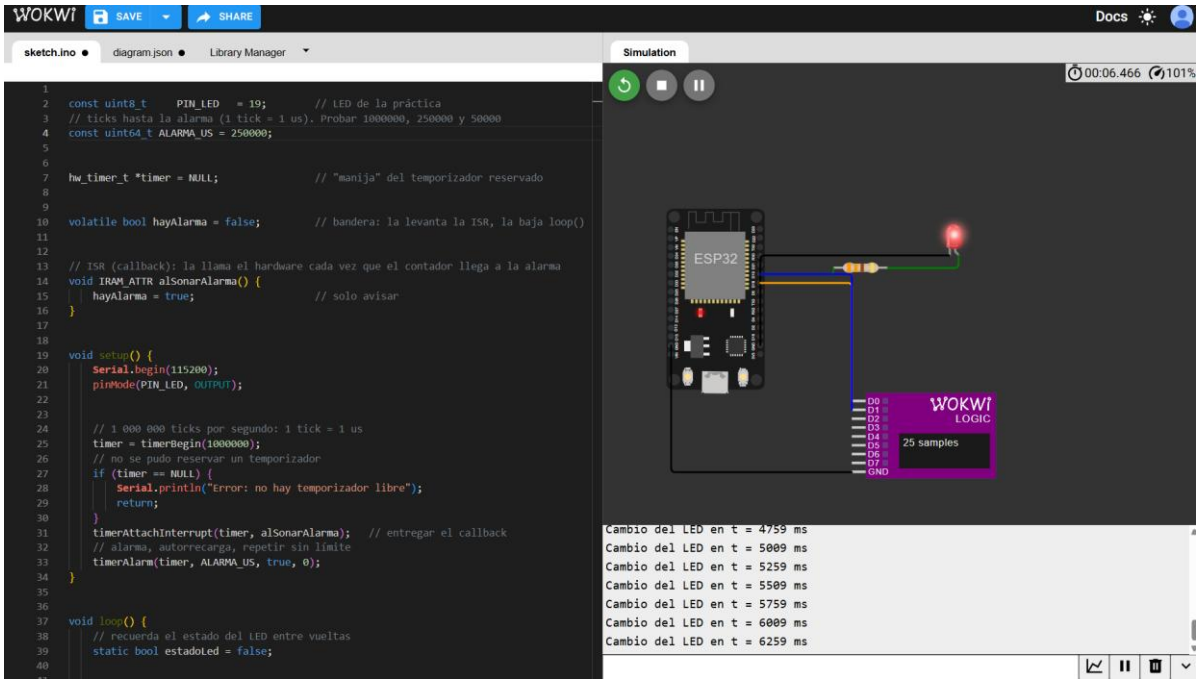
A7. Detengan la simulación. Cambien `ALARMA_US` a 250000 y repitan los pasos A4 a A6. Luego hagan lo mismo con 50000.

A8. Tomen una captura de pantalla del Monitor Serie para cada valor.

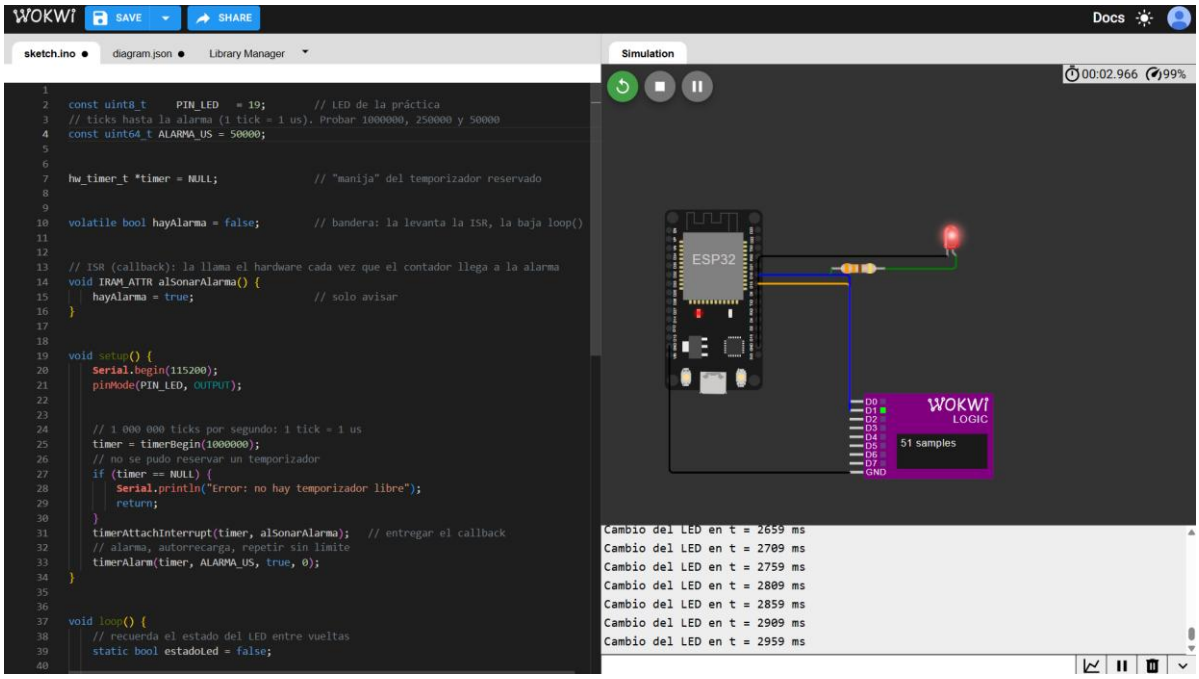
1000000



250000



50000




```
const uint8_t     PIN_LED   = 19;        // LED de la práctica
// ticks hasta la alarma (1 tick = 1 us). Probar 1000000, 250000 y 50000
const uint64_t ALARMA_US = 50000;


hw_timer_t *timer = NULL;               // "manija" del temporizador reservado


volatile bool hayAlarma = false;        // bandera: la levanta la ISR, la baja loop()


// ISR (callback): la llama el hardware cada vez que el contador llega a la alarma
void IRAM_ATTR alSonarAlarma() {
    hayAlarma = true;                   // solo avisar
}


void setup() {
    Serial.begin(115200);
    pinMode(PIN_LED, OUTPUT);


    // 1 000 000 ticks por segundo: 1 tick = 1 us
    timer = timerBegin(1000000);
    // no se pudo reservar un temporizador
    if (timer == NULL) {
        Serial.println("Error: no hay temporizador libre");
        return;
    }
    timerAttachInterrupt(timer, alSonarAlarma);   // entregar el callback
    // alarma, autorrecarga, repetir sin límite
    timerAlarm(timer, ALARMA_US, true, 0);
}


void loop() {
    // recuerda el estado del LED entre vueltas
    static bool estadoLed = false;
```

Código 1. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías · pin GPIO19

```
const uint8_t  PIN_LED    = 19;         // LED de la práctica
// ticks hasta la alarma (1 tick = 1 us). Probar 1000000, 250000 y 50000
const uint64_t ALARMA_US = 1000000;

hw_timer_t *timer = NULL;               // "manija" del temporizador reservado

volatile bool hayAlarma = false;        // bandera: la levanta la ISR, la baja loop()

// ISR (callback): la llama el hardware cada vez que el contador llega a la alarma
void IRAM_ATTR alSonarAlarma() {
  hayAlarma = true;                     // solo avisar
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);

  // 1 000 000 ticks por segundo: 1 tick = 1 us
  timer = timerBegin(1000000);
  // no se pudo reservar un temporizador
  if (timer == NULL) {
    Serial.println("Error: no hay temporizador libre");
    return;
  }
  timerAttachInterrupt(timer, alSonarAlarma);  // entregar el callback
  // alarma, autorrecarga, repetir sin límite
  timerAlarm(timer, ALARMA_US, true, 0);
}

void loop() {
  // recuerda el estado del LED entre vueltas
  static bool estadoLed = false;

  if (hayAlarma) {                              // ¿está levantada la bandera?
    hayAlarma = false;                          // bajarla
    estadoLed = !estadoLed;                     // invertir el estado
    digitalWrite(PIN_LED, estadoLed);           // cambiar el LED
    Serial.printf("Cambio del LED en t = %lu ms\n", millis());
  }
}
```

`Serial.printf` imprime texto con valores dentro: `%lu` se reemplaza por el número que va después de la coma, y `\n` salta de línea.

Tabla A. Tiempo entre cambios del LED

ALARMA_US (ticks)	Tiempo calculado entre cambios (ms)	Tiempo medido en el Monitor Serie (ms)	Periodo completo del parpadeo (ms)
1 000 000	1000	1000	2000
250 000	250	250	500
50 000	50	50	100

El periodo completo del parpadeo es el tiempo que tarda el LED en encenderse y volver a encenderse: dos cambios.

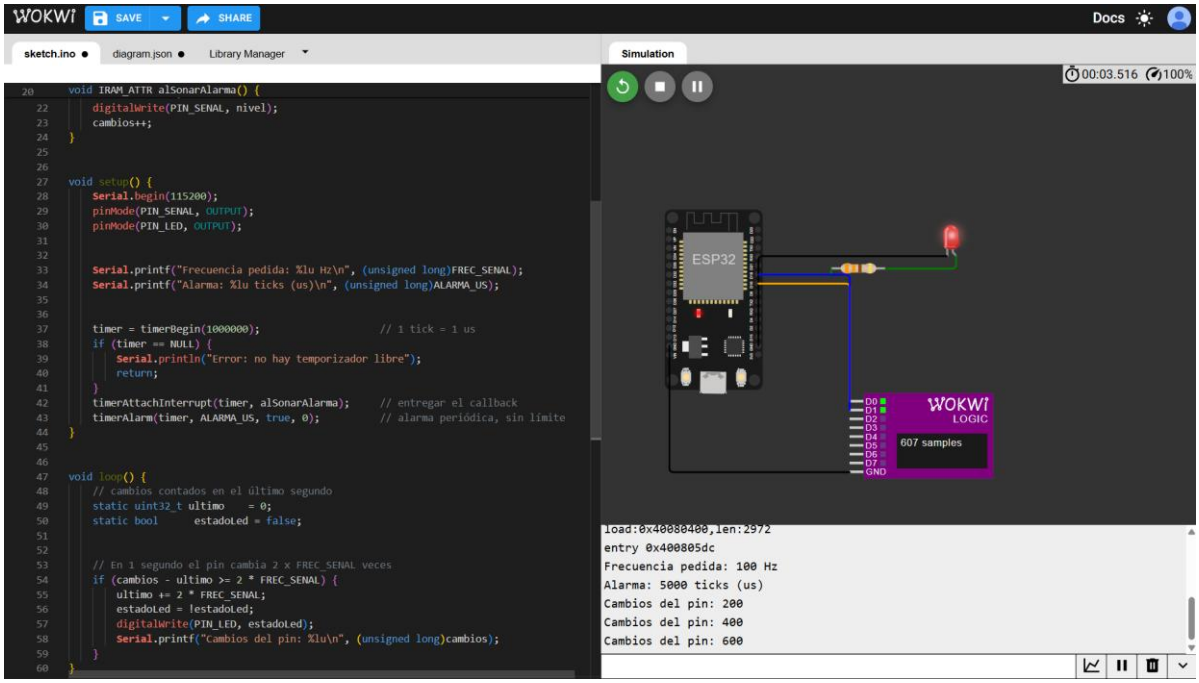
Parte B. Generar y medir una onda cuadrada (Wokwi, 50 min)

En esta parte se escribe la frecuencia deseada en `FREC_SENAL` y el programa calcula la alarma con la fórmula del factor 2. En cada alarma, la ISR invierte el GPIO18 y cuenta el cambio; cada segundo, `loop()` cambia el LED del GPIO19 (latido) e imprime cuántos cambios lleva el pin. Esta vez el pin sí se cambia dentro de la ISR: es una operación muy corta y así el cambio ocurre en el instante exacto de la alarma, sin esperar a `loop()`.

B1. En el mismo proyecto, reemplacen el contenido de sketch.ino por el Código 2. El diagram.json no cambia: el canal D0 del analizador ya está conectado al GPIO18.

B2. Con `FREC_SENAL = 100`, calculen a mano `ALARMA_US` con la fórmula del factor 2 (Fundamento teórico) y anótenlo en la Tabla B.

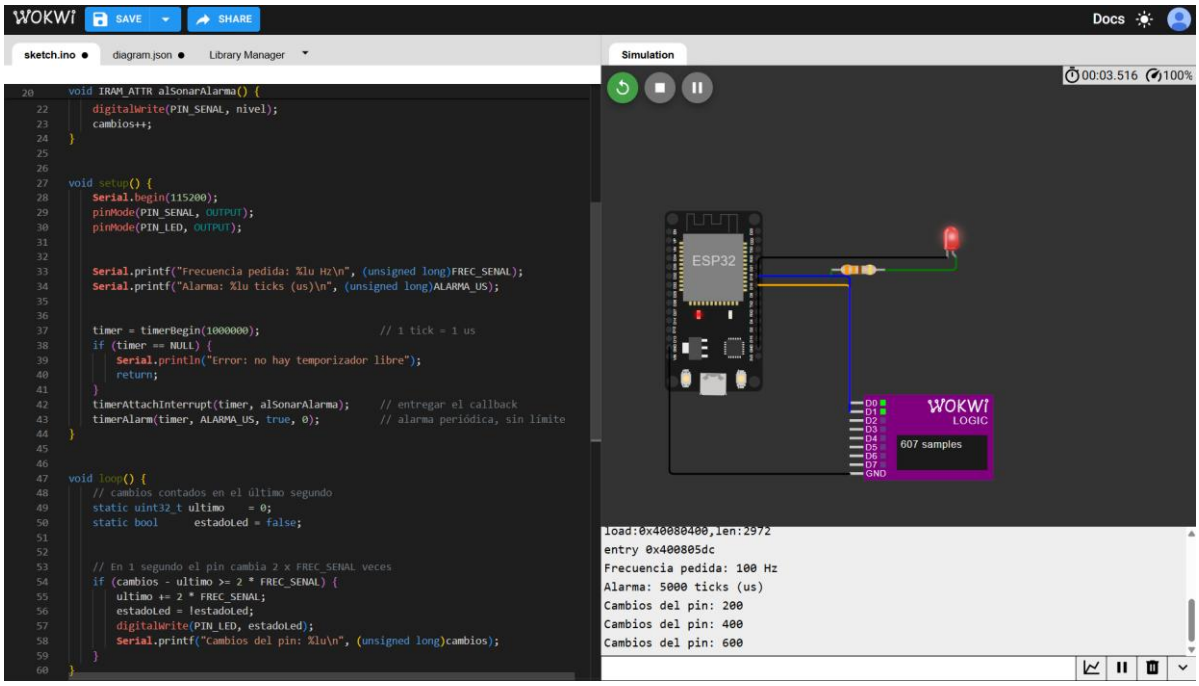
B3. Presionen ▶. Comprueben que la alarma que imprime el Monitor Serie coincide con su cálculo y que el LED parpadea una vez por segundo.



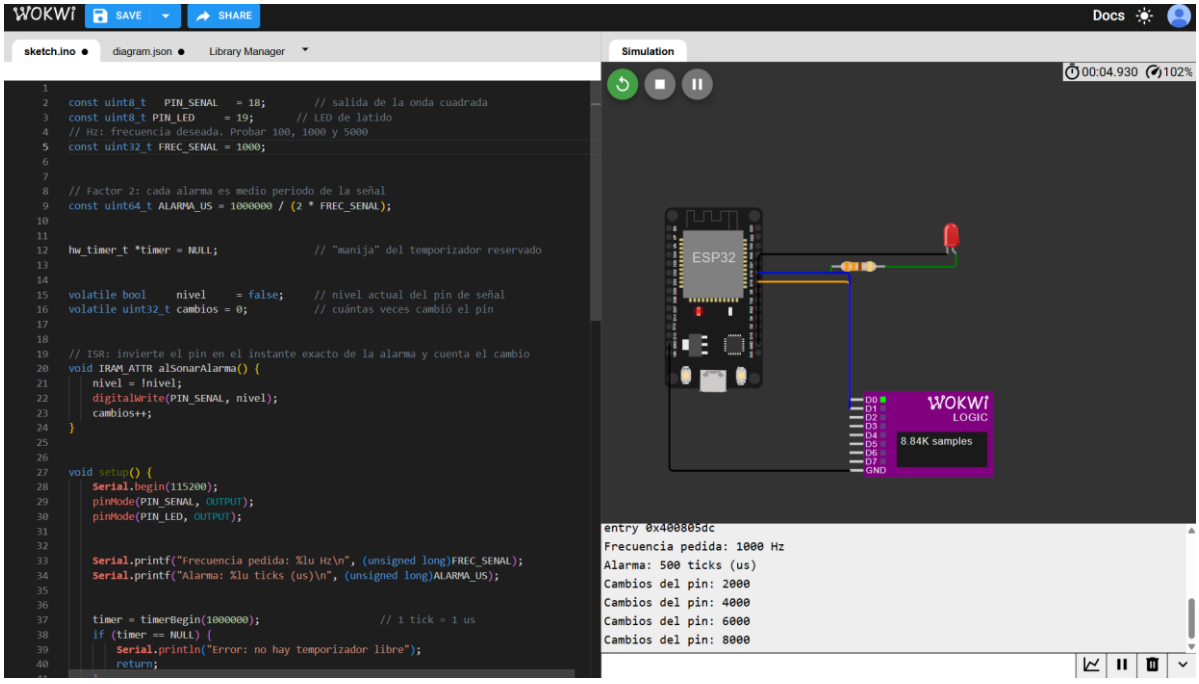
B4. El Monitor Serie imprime cada segundo cuántos cambios lleva el pin. Resten dos valores consecutivos: eso son los cambios por segundo. La frecuencia medida es la mitad de ese número (cada ciclo tiene dos cambios). Anoten ambos valores en la Tabla B.

B5. Detengan la simulación y repitan los pasos B2 a B4 con `FREC_SENAL = 1000` y luego con `FREC_SENAL = 5000`.

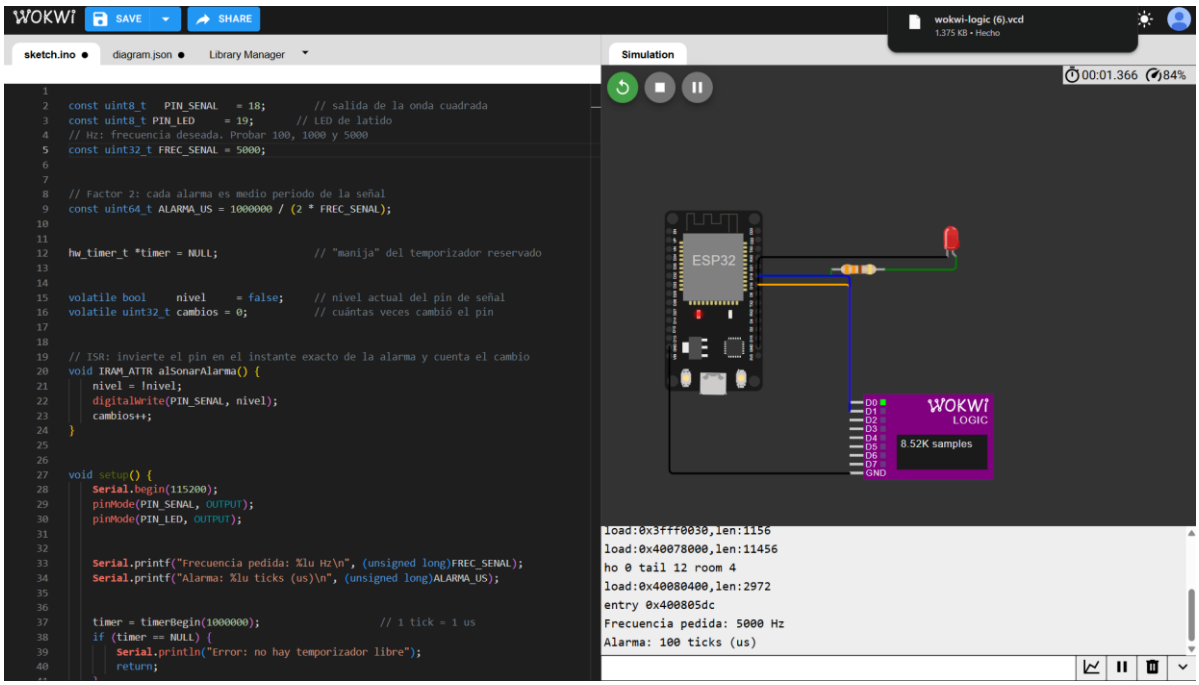
100



1000



5000



B6. Medición con el analizador, solo para 1000 Hz: pongan `FREC_SENAL = 1000`, inicien la simulación, déjenla correr unos 3 segundos y deténganla con el botón de parar. Wokwi descargará el archivo `wokwi-logic.vcd`.

B7. Abran el archivo en PulseView (Open → flecha a la derecha → Import Value Change Dump data). En la señal logic.D0, acerquen la vista y midan el periodo: el tiempo entre dos flancos de subida consecutivos (dos momentos en que la señal pasa de bajo a alto). Tomen una captura de pantalla y anoten el periodo en la Tabla B.

Código 2. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías · pines GPIO18 (señal) y GPIO19 (LED de latido)

```
const uint8_t  PIN_SENAL  = 18;        // salida de la onda cuadrada
const uint8_t  PIN_LED    = 19;        // LED de latido
// Hz: frecuencia deseada. Probar 100, 1000 y 5000
const uint32_t FREC_SENAL = 100;

// Factor 2: cada alarma es medio periodo de la señal
const uint64_t ALARMA_US = 1000000 / (2 * FREC_SENAL);

hw_timer_t *timer = NULL;              // "manija" del temporizador reservado

volatile bool     nivel   = false;     // nivel actual del pin de señal
volatile uint32_t cambios = 0;         // cuántas veces cambió el pin
```

```
// ISR: invierte el pin en el instante exacto de la alarma y cuenta el cambio
void IRAM_ATTR alSonarAlarma() {
  nivel = !nivel;
  digitalWrite(PIN_SENAL, nivel);
  cambios++;
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_SENAL, OUTPUT);
  pinMode(PIN_LED, OUTPUT);

  Serial.printf("Frecuencia pedida: %lu Hz\n", (unsigned long)FREC_SENAL);
  Serial.printf("Alarma: %lu ticks (us)\n", (unsigned long)ALARMA_US);

  timer = timerBegin(1000000);                  // 1 tick = 1 us
  if (timer == NULL) {
    Serial.println("Error: no hay temporizador libre");
    return;
  }
  timerAttachInterrupt(timer, alSonarAlarma);  // entregar el callback
  timerAlarm(timer, ALARMA_US, true, 0);       // alarma periódica, sin límite
}

void loop() {
  // cambios contados en el último segundo
  static uint32_t ultimo    = 0;
  static bool     estadoLed = false;

  // En 1 segundo el pin cambia 2 x FREC_SENAL veces
  if (cambios - ultimo >= 2 * FREC_SENAL) {
    ultimo += 2 * FREC_SENAL;
    estadoLed = !estadoLed;
    digitalWrite(PIN_LED, estadoLed);
    Serial.printf("Cambios del pin: %lu\n", (unsigned long)cambios);
  }
}
```

`(unsigned long)` convierte el número al tipo que espera `%lu`, para que se imprima bien.

Tabla B. Generación y medición de la onda cuadrada

FREC_SENAL pedida (Hz)	ALARMA_US calculada a mano	ALARMA_US impresa por el programa	Cambios por segundo (Monitor Serie)	Frecuencia medida = cambios / 2 (Hz)	Periodo medido en PulseView (ms)
100	5000	200	200	100	No se mide
1000	500	2000	2000	1000	1ms
5000	100	10000	10000	5000	No se mide

Si PulseView no está instalado o no abre el archivo, dejen vacía la última columna y explíquenlo en el informe; las frecuencias medidas con el Monitor Serie son suficientes para el resto de la tabla.

Parte C. Prueba en la placa ESP32 real (grupos con placa, 20 min)

Se carga en la placa el Código 2 con `FREC_SENAL = 1000`. Los grupos sin placa no realizan esta parte y no pierden puntos (ver Evaluación); durante este tiempo adelantan las preguntas de control.

C1. Con el USB desconectado, armen en la protoboard el LED con su resistencia en el GPIO19, según la Tabla de conexiones. Revisen la polaridad: pata larga hacia la resistencia, pata corta hacia GND.

C2. Conecten el cable USB. En Arduino IDE elijan Herramientas → Placa → ESP32 Dev Module y el puerto correcto. Carguen el Código 2 con `FREC_SENAL = 1000`.

C3. Abran el Monitor Serie a 115200 baudios y comprueben que el LED parpadea una vez por segundo.

C4. Si hay osciloscopio o multímetro con modo Hz: conecten la punta al GPIO18 y la referencia a GND. Midan la frecuencia y tómenle una foto.

C5. Anoten los resultados en la Tabla C.

Tabla C. Medición en la placa real

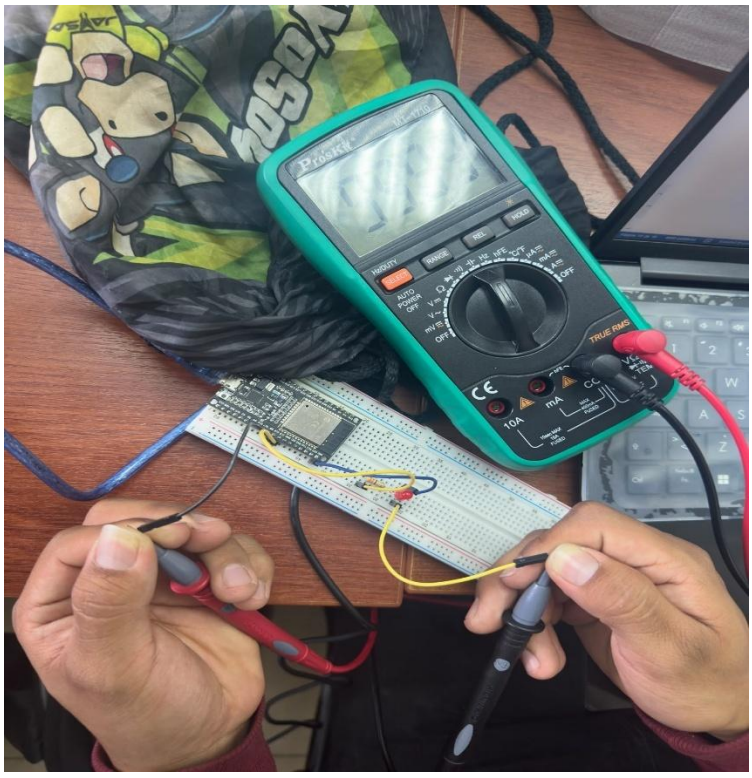
Instrumento usado	Frecuencia pedida (Hz)	Frecuencia medida (Hz)	Observaciones
Multimetro	100	99.99Hz	Lectura precisa con un error del 1%
Multimetro	1000	99.99Hz	Lectura precisa con un error del 1%
Multimetro	5000	4.99Hz	Lectura precisa con un error del 1%

Si algo falla, revisen el Anexo B (Solución de problemas) antes de pedir ayuda al técnico docente.

100



1000



5000



Parte D. Entrega (10 min)

D1. Suban al repositorio de GitHub del grupo una carpeta AP3 con:

- los dos códigos (`.ino`);
- el `diagram.json`;
- el archivo `wokwi-logic.vcd`;
- las capturas (Monitor Serie, PulseView y, si hubo Parte C, la foto de la medición).

D2. Suban al EVA, en la tarea de la AP3, el informe en PDF con el enlace al repositorio.

D3. Plazo: el que indique el docente en el EVA.

7. Resultados esperados:

- **Código 1:** el LED cambia de estado en cada alarma y el Monitor Serie imprime el instante de cada cambio; la diferencia entre dos instantes consecutivos coincide con el tiempo calculado a partir de `ALARMA_US` (Tabla A).
- **Código 2:** el Monitor Serie imprime una alarma igual a la calculada con la fórmula del factor 2, el LED de latido parpadea una vez por segundo y la frecuencia medida (cambios / 2) coincide con `FREC_SENAL` (Tabla B).
- **Analizador lógico:** con `FREC_SENAL = 1000`, la señal logic.D0 en PulseView muestra una onda cuadrada de 1 ms de periodo.
- **Placa real (Parte C):** el mismo comportamiento que en Wokwi y una frecuencia medida cercana a 1000 Hz (Tabla C).

Se debe entregar el esquema del circuito, los dos códigos, el archivo .vcd, las capturas y las Tablas A, B y C completas.

8. Preguntas de Control:

Respondan en el informe, con sus palabras y mostrando los cálculos.

1. En el Código 1, la ISR solo pone `hayAlarma = true`. ¿Por qué no se imprime el mensaje directamente dentro de la ISR?

Porque la ISR debe ejecutarse lo más rápido posible para no interrumpir durante demasiado tiempo el funcionamiento del microcontrolador. Imprimir mensajes mediante Serial.println() requiere más tiempo y puede ocasionar retrasos. Por esta razón, la interrupción únicamente activa una bandera y loop() se encarga de mostrar el mensaje en el Monitor Serie.

2. Con `timerBegin(1000000)` y `ALARMA_US = 250000`, ¿cada cuánto cambia el LED? ¿Cuánto dura el periodo completo del parpadeo? Comparen con su Tabla A.

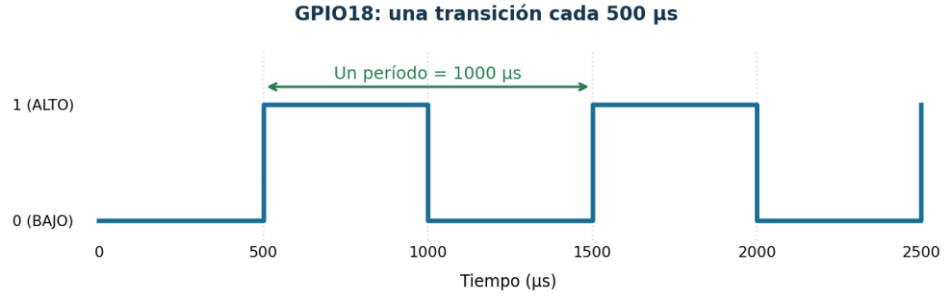
Con una frecuencia de 1 000 000 ticks por segundo, cada tick equivale a 1 µs. El tiempo entre cambios se obtiene dividiendo los ticks de alarma entre la frecuencia del temporizador:
t = 250 000 / 1 000 000 = 0,25 s = 250 ms
Como un parpadeo completo necesita dos cambios (encendido y apagado), su periodo es:
T = 250 ms × 2 = 500 ms
Al comparar los cálculos con la Tabla A, se observa que, para ALARMA_US = 250 000, el tiempo calculado entre cambios es de 250 ms y el tiempo medido en el Monitor Serie también es de 250 ms. La diferencia entre ambos es 250 − 250 = 0 ms, equivalente a un error del 0 %. Como el LED necesita dos cambios para completar un ciclo (encenderse y apagarse), el periodo completo es 250 × 2 = 500 ms, que coincide con los 500 ms registrados en la Tabla A. Por tanto, la simulación confirma el funcionamiento esperado del temporizador.

3. Si se quita la palabra `volatile` de la variable `hayAlarma`, ¿qué podría pasar y por qué?

Si eliminamos volatile, el compilador podría optimizar el programa suponiendo que la variable no cambia inesperadamente. Como hayAlarma se modifica dentro de una interrupción, loop() podría no detectar correctamente su nuevo valor, provocando que el LED no cambie cuando corresponde. Por eso se utiliza volatile, para indicar que la variable puede modificarse fuera del flujo normal del programa.

4. Expliquen con un dibujo por qué, con una alarma cada 500 µs, la señal del GPIO18 tiene 1 kHz y no 2 kHz.

Cada alarma provoca un cambio en el estado del GPIO18. Sin embargo, para completar un ciclo de la onda cuadrada se necesitan dos cambios: pasar de bajo a alto y regresar de alto a bajo.



T = 500 µs + 500 µs = 1000 µs = 0,001 s

f = 1 / T = 1 / 0,001 = 1000 Hz = 1 kHz

La alarma ocurre 2000 veces por segundo, pero la señal completa solamente 1000 ciclos. Por eso la frecuencia de la onda cuadrada es 1 kHz y no 2 kHz.

5. Calculen `ALARMA_US` para una señal de 2 kHz y para una de 250 Hz.

Se utiliza la fórmula ALARMA_US = 1 000 000 / (2 × f), porque cada alarma representa medio periodo de la señal.

Para 2 kHz (2000 Hz): 1 000 000 / (2 × 2000) = 250 ticks

Para 250 Hz: 1 000 000 / (2 × 250) = 2000 ticks

Por lo tanto, se necesita una alarma cada 250 µs para generar una señal de 2 kHz y cada 2000 µs para una señal de 250 Hz.

6. ¿Coinciden las frecuencias medidas en la Tabla B con las pedidas? Si hay diferencias, ¿a qué las atribuyen?

Sí. De acuerdo con la Tabla B, las frecuencias medidas coinciden con las pedidas. Para

100 Hz se registraron 200 cambios por segundo, de modo que f = 200 / 2 = 100 Hz; para 1000 Hz hubo 2000 cambios por segundo, por lo que f = 2000 / 2 = 1000 Hz; y para 5000 Hz se obtuvieron 10 000 cambios por segundo, de manera que f = 10 000 / 2 = 5000 Hz. En los tres casos la diferencia es de 0 Hz y el error porcentual registrado es del 0 %. Asimismo, el periodo de 1 ms indicado para 1000 Hz concuerda con T = 1 / 1000 s. Por eso, según los datos consignados, no existen diferencias que deban atribuirse a fallos de medición. Sin embargo, en la columna «ALARMA_US impresa por el programa» se anotaron 200, 2000 y 10 000, valores que coinciden con los cambios por segundo y no con la alarma: según el Código 2, los valores de alarma para esas tres frecuencias son 5000, 500 y 100 ticks, respectivamente. Conviene revisar esa columna antes de entregar el informe.

7. Con `timerBegin(1000000)`, ¿qué valor de prescaler N calcula el core? Muestren el cálculo a partir del reloj de 80 MHz.

El prescaler permite dividir la frecuencia del reloj para obtener la frecuencia con la que trabajará el temporizador. Se parte de un reloj de 80 MHz (80 000 000 Hz) y se busca una frecuencia de 1 000 000 Hz.

N = f_reloj / f_temporizador

N = 80 000 000 / 1 000 000 = 80

El prescaler calculado es 80. Con esta división, el temporizador cuenta 1 000 000 ticks por segundo, es decir, un tick cada microsegundo.

8. En esta práctica, ¿por qué el contador del temporizador nunca llega a desbordarse?

El prescaler permite dividir la frecuencia del reloj para obtener la frecuencia con la que trabajará el temporizador. Se parte de un reloj de 80 MHz (80 000 000 Hz) y se busca una frecuencia de 1 000 000 Hz.
N = f_reloj / f_temporizador
N = 80 000 000 / 1 000 000 = 80
El prescaler calculado es 80. Con esta división, el temporizador cuenta 1 000 000 ticks por segundo, es decir, un tick cada microsegundo.

9. Entregables y estructura del informe

El informe debe tener la siguiente estructura:

- Portada con los datos de la UNL, los nombres de la pareja, el paralelo, la fecha y el enlace al repositorio.
- Objetivo.
- Esquema del circuito: captura del diagrama de Wokwi y, si aplica, foto del montaje real.
- Resultados: Tablas A, B y C completas, con las capturas del Monitor Serie y de PulseView o GTKWave.
- Respuestas a las preguntas de control.
- Conclusiones: tres como máximo, basadas en sus mediciones.
- Declaración de uso de IA generativa: qué herramienta usaron, para qué y qué parte del informe se vio afectada; si no usaron ninguna, declárenlo.

Entrega: la carpeta AP3 en el repositorio de GitHub del grupo (Parte D) y, en la tarea de la AP3 del EVA, el informe en PDF con el enlace al repositorio.

10. Evaluación

Calificación sobre 10 puntos. Cada nivel vale el porcentaje indicado de los puntos del criterio. Los grupos sin placa no pierden puntos por la Parte C: esa parte no se califica y la nota se calcula sobre 9 puntos y se lleva a 10 (nota × 10 / 9).

Criterio	Puntos	Excelente (100 %)	Satisfactorio (70 %)	En desarrollo (40 %)	No presenta (0 %)

Parte A: Tabla A con predicción y medición	2	Los tres valores calculados y medidos coinciden y el periodo completo está bien deducido	Tabla completa con capturas, pero un valor o el periodo completo está mal deducido	Tabla incompleta o sin capturas	No presenta
Parte B: Tabla B y capturas de la señal	3	Las tres frecuencias calculadas y medidas, y la	Las tres frecuencias medidas; falta la captura de	Solo una o dos frecuencias medidas, o sin capturas	Sin mediciones ni capturas

		captura de PulseView a 1000 Hz con el periodo correcto	PulseView o el periodo es incorrecto		
Parte C: medición en placa real	1	Montaje funcionando y Tabla C completa	Montaje funcionando; Tabla C incompleta	Montaje con fallas o sin evidencia de la medición	Sin evidencia
Preguntas de control	2	Las 8 respuestas correctas, con cálculos	6 o 7 correctas	3 a 5 correctas	Menos de 3
Repositorio de GitHub	1	Carpeta AP3 con todos los archivos pedidos	Falta un archivo	Faltan varios archivos	Sin repositorio
Informe y declaración de uso de IA	1	Estructura completa, conclusiones basadas en datos y declaración de IA	Conclusiones genéricas o falta un apartado	Faltan varios apartados	Falta la declaración o la estructura

La retroalimentación se entregará en un máximo de 5 días hábiles.

11. Bibliografía

1. Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.
2. Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.
3. Espressif Systems. (s. f.). ESP32 Series Datasheet. https://www.espressif.com/en/support/documents/technical-documents
4. Espressif Systems. (s. f.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/
5. CodeMagic LTD. (2025). Wokwi – documentación. https://docs.wokwi.com
6. Arduino. (2024). Arduino Language Reference. https://www.arduino.cc/reference/
7. sigrok. (s. f.). PulseView. https://sigrok.org/wiki/PulseView
8. GTKWave. (s. f.). GTKWave – documentación. https://gtkwave.sourceforge.net/

12. Elaboración y Aprobación

Elaborado por	Ing. Luis Darío Sinche Cueva **Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: LUIS DARIO SINCHE CUEVA
Revisado por	Ing. Ramiro Rene Rivera Guamán **Técnico Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Ing. Edison Leonardo Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROM ER Validar únicamente con FirmaEC

Anexo A. diagram.json de Wokwi

Peguen este contenido en la pestaña diagram.json (paso A2).

```
{
  "version": 1,
  "author": "Sistemas Digitales UNL",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {}
},
    { "type": "wokwi-resistor", "id": "r1", "top": 60, "left": 180, "attrs": {
"value": "330" } },
    { "type": "wokwi-led", "id": "led1", "top": 10, "left": 290, "attrs": { "color":
"red" } },
    { "type": "wokwi-logic-analyzer", "id": "logic1", "top": 200, "left": 200,
"attrs": { "channelNames": "SENAL,LED" } }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:D19", "r1:1", "green", [] ],
    [ "r1:2", "led1:A", "green", [] ],
    [ "led1:C", "esp:GND.1", "black", [] ],
    [ "esp:D18", "logic1:D0", "orange", [] ],
    [ "esp:D19", "logic1:D1", "blue", [] ],
    [ "logic1:GND", "esp:GND.2", "black", [] ]
  ]
}
```

Anexo B. Solución de problemas

Antes de pedir ayuda al técnico docente, revisen esta tabla.

Síntoma	Causa probable	Qué hacer
Error de compilación: «too many arguments» o «was not declared in this scope» en funciones timer…	Código copiado de internet, escrito para la versión 2 del core	Usen solo los Códigos 1 y 2 de esta guía
En el Arduino IDE no aparece «ESP32 Dev Module»	No está instalado el core del ESP32	Herramientas → Placa → Gestor de placas → buscar «esp32» → instalar «esp32 by Espressif Systems» (versión 3.x)
El Monitor Serie muestra caracteres raros	Velocidad distinta de la del código	Pongan el Monitor Serie en 115200 baudios
El Monitor Serie dice «Error: no hay temporizador libre»	No se pudo reservar un temporizador	Reinicien la simulación o presionen EN en la placa
El LED no enciende en Wokwi	El diagram.json no se pegó completo	Borren todo el contenido de diagram.json y peguen de nuevo el Anexo A
El LED no enciende en la placa	LED invertido o pin equivocado	Pata larga hacia la resistencia; resistencia al GPIO19 (D19)
No se descarga el archivo .vcd	La simulación no se detuvo con el botón de parar, o el navegador bloqueó la descarga	Detengan con el botón de parar de Wokwi y permitan las descargas del sitio
PulseView no abre el archivo	Se usó Open en lugar de importar	Open → flecha a la derecha → Import Value Change Dump data
La frecuencia medida es la mitad de la esperada	Se olvidó el factor 2 al calcular	Revisen el factor 2 en el Fundamento teórico

No pueden abrir el .vcd de ninguna forma	—	Usen la columna «Cambios por segundo» de la Tabla B y explíquenlo en el informe
Error de compilación con la palabra «stray» después de pegar el código	Al copiar desde la guía se pegó algún carácter especial (comillas curvas o un guion largo)	Vayan a la línea que indica el error y vuelvan a escribir a mano las comillas " o el carácter extraño
Wokwi muestra un error al pegar el diagram.json	Falta una llave, un corchete o una comilla al copiar	Copien de nuevo el Anexo A completo, desde la primera llave { hasta la última }

Enlace git: https://github.com/Orojco/Orozco-Marco.git